

The **I-Wallet** is a decentralized infrastructure protocol designed to provide autonomous AI agents with a secure, on-chain identity and financial execution layer. By leveraging **Zero-Knowledge (ZK) Proofs** and **Sui's Object-Centric architecture**, the protocol eliminates the need for agents to hold long-lived private keys, effectively neutralizing risks like key extraction and financial prompt injection.

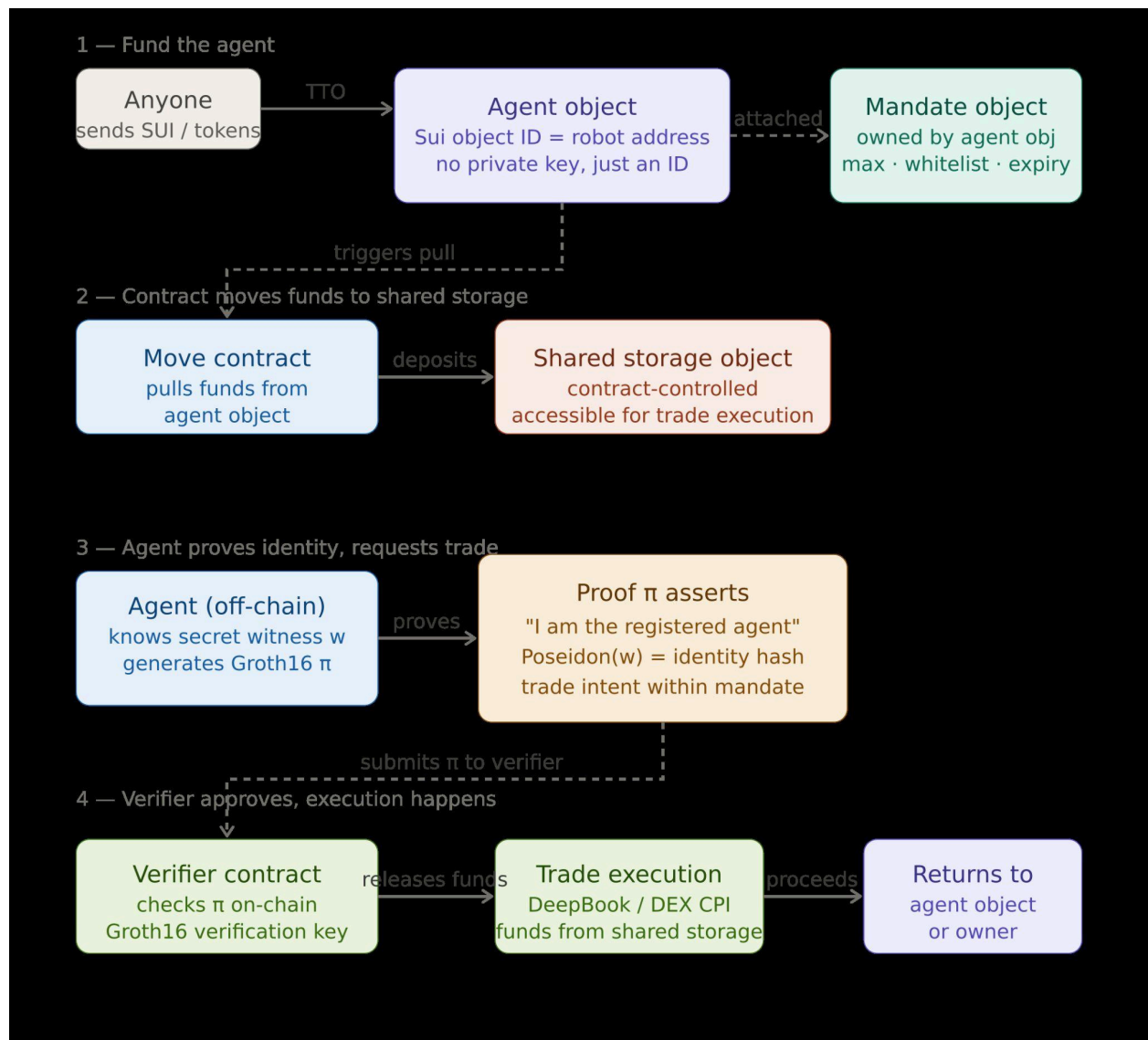
The Problem

Existing wallet solutions are built for human-in-the-loop interactions. When applied to AI agents, they create significant security vulnerabilities:

- **Key Exposure:** Agents must store private keys in hot environments to sign transactions autonomously.
 - **Trust Gaps:** There is no cryptographic way to ensure an agent's action aligns with the owner's intent.
 - **Infrastructure Friction:** Legacy wallets rely on manual approval GUIs, which throttle machine-speed execution.
-

The Solution: Robot Account Abstraction

I-Wallet moves the industry away from "AI with a Private Key" toward **"AI with a Verifiable Identity."** It functions as a specialized account abstraction layer where the wallet is an on-chain object governed by mathematical proofs rather than digital signatures.



The actual security model — in plain terms

The "no private key" claim is true and defensible, but for a different reason than the critique assumed. The flow is:

1. **Funds go to an object ID**, not to an address controlled by a signing key. The agent object *is* its own Sui object — it receives assets via TTO.
2. **The contract, not the agent, controls fund movement.** When a trade is needed, the Move contract pulls funds from the agent object into a shared storage object. The agent never directly holds a signing key to authorize this — the contract logic *is* the authorization.

3. **ZK proves the caller is the legitimate agent.** The Groth16 proof answers: "is the entity requesting this trade the registered agent for this object?" If yes, the verifier unlocks the shared storage funds for the execution.

So "no private key" means: **no EOA-style key that can drain funds arbitrarily**. The only thing that can move money is the contract, and the contract only moves money when a valid Groth16 proof is presented.

ZK-Authorization Engine

The ZK proof does one specific job: it proves the caller is the legitimate registered agent without revealing the agent's secret identifier. Using Groth16, the agent computes a proof that `Poseidon(w) == registered_identity_hash` where `w` is a secret known only to the agent daemon. The verifier contract checks this proof on-chain before releasing funds from shared storage for any trade. This means even if the agent's server is fully compromised, an attacker cannot drain funds — they would need to forge a valid Groth16 proof, which is computationally infeasible.

Private input: `w` (agent secret)

Public input: `identity_hash = Poseidon(w)`, registered on-chain at agent creation

Constraint: `Poseidon(w) == identity_hash`

Mandate checks: amount, target contract, expiry — enforced by the Move verifier contract reading the Mandate object directly, not inside the circuit

I-Wallet High-Velocity Roadmap (46 Days)

Phase 1: The "Identity" Sprint (May 13 – May 24)

- **Goal:** Establish the ZK-circuit and the `AgentObject` Move logic.
- **Critical Path:** Successfully verify a Groth16 proof on the Sui Testnet using the `sui::groth16` module.

Phase 2: The "Vault" Sprint (May 25 – June 7)

- **Goal:** Build the "Pull" mechanism and `MandateObject` enforcement.
- **Critical Path:** Move funds from an `AgentObject` to `Shared Storage` based on a valid ZK proof and Mandate check.

Phase 3: The "Ecosystem" Sprint (June 8 – June 21)

- **Goal:** Integration with DeepBook for the demo and Walrus for the audit trail.
- **Critical Path:** Agent successfully executes a trade on DeepBook and logs the intent/proof hash to Walrus.

Phase 4: The "Submission" Sprint (June 22 – June 28)

- **Goal:** Final UI polish, documentation, and the "Pitch Video."
- **Critical Path:** Complete the "Trojan Horse" demo (AI Trading Agent) and submit to Sui Overflow.

Task Breakdown for the "War Room"

1. Protocol Engineer (George-Goldman)

- **Circuit Optimization:** Finalize the Groth16 circuit for `$Poseidon(w) == identity\_hash$` using an existing parameter set to save time.
- **Move Verifier:** Code the core `validate_and_execute` function in Move, integrating the `sui::groth16` module.
- **Cryptography:** Ensure the intent-binding logic (Target + Amount + Nonce) is correctly hashed as a public input.

2. Sui Solution Engineer

- **Object Architecture:** Implement the `AgentObject`, `MandateObject`, and `SharedStorage` as detailed in `i_wallet_actual_architecture.jpg`.
- **Integration:** Map the `Trade Execution` logic to **DeepBook** V3's CPI calls.
- **Infrastructure:** Connect the execution logs to **Walrus** to create the verifiable audit trail.

3. Front-End Engineer

- **The "Provisioner":** Build a minimalist dashboard to generate the secret `$w$`, hash it, and create the `AgentObject`.
- **Mandate Control:** Create a UI for humans to set spending limits and whitelist DeepBook pairs.
- **Real-time Feed:** Display the agent's balance and its Walrus-backed activity log.

4. Dev Advocate / Community Guy

- **"No Private Key" Docs:** Draft the technical documentation explaining the security model in plain terms for the judges.
- **The Pitch:** Write the script and coordinate the 3-minute demo video emphasizing "Sui as the Native Execution Layer for AI."
- **Submission Copy:** Handle the GitBook/README and ensure the Sui Overflow application highlights the "Built to Last" criteria.

5. Project Manager

- **The Demo (Trojan Horse):** Oversee the development of the AI Trading Agent script (in Rust or Go) that uses the I-Wallet SDK to generate proofs.
- **Milestone Tracking:** Daily syncs to ensure Phase 1 (On-chain ZK verification) is complete by May 24.

- **Submission Quality:** Ensure the project hits the **Agentic Web**, **Walrus**, and **DeepBook** specialized tracks for maximum prize surface area.